
Aula prática 1 — Contagem de palavras com Map/Reduce

Linux, Shell e C numa aplicação sequencial

Charles Ferreira Gonçalves

2026-09-21

Índice

1	Aula prática 1 — Contagem de palavras	2
1.1	Linux, Shell e C através de Map/Reduce	2
1.2	Objetivos	2
1.3	Preparação	2
1.4	1. O modelo Map/Reduce	3
1.5	2. Conhecer os dados	4
1.6	3. Construir o pipeline Linux	5
1.7	4. Automatizar o pipeline com Bash	6
1.8	5. Reimplementar em C	7
1.8.1	5.1 O tipo usado	8
1.8.2	5.2 Pseudocódigo	9
1.9	6. Make e validação	9
1.9.1	Casos cobertos	10
1.10	7. Observação opcional com GDB	10
1.11	8. Trabalho individual	11
1.12	Checklist	11

1 Aula prática 1 — Contagem de palavras

1.1 Linux, Shell e C através de Map/Reduce

Nesta aula vamos resolver o mesmo problema de três formas: com comandos Linux, com um script Bash e com um programa C.

O problema é simples:

Dado um ficheiro de texto, mostrar as N palavras mais frequentes.

O resultado final será uma aplicação sequencial. Não vamos ainda usar processos, threads ou comunicação entre processos.

1.2 Objetivos

No fim da aula deverá conseguir:

- navegar no sistema de ficheiros e inspecionar um ficheiro;
- construir um pipeline com pipes e redirecionamento;
- explicar map, agrupamento e reduce neste problema;
- escrever e executar um script Bash simples;
- compilar um programa C que lê um ficheiro;
- completar a função de redução fornecida;
- usar make e comparar duas implementações com `diff`.

1.3 Preparação

Esta aula funciona na Linux VM, no Google Cloud Shell e no macOS com as ferramentas POSIX instaladas.

A partir da raiz da cópia da aula (PL1), execute o script de validação:

```
bash scripts/validate_environment.sh
```

Se algum comando necessário não existir, peça ajuda antes de começar. Não gaste o tempo da aula a configurar o ambiente.

Entre na pasta de trabalho do aluno:

```
cd src
chmod +x wordcount.sh
```

Os dados estão em `../data/`.

Important

A cópia entregue aos alunos contém `src/`. Não é necessário acesso a uma implementação de referência para realizar esta aula.

1.4 1. O modelo Map/Reduce

Considere:

Linux, linux! memory.

O **map** separa e normaliza as palavras:

```
(linux, 1)
(linux, 1)
(memory, 1)
```

O agrupamento reúne as chaves iguais:

```
linux    -> [1, 1]
memory   -> [1]
```

O **reduce** combina os valores:

```
linux    -> 2
memory   -> 1
```

Nesta aula, `shuffle/group` é apenas ordenar ou agrupar dados na mesma execução. Não há máquinas remotas nem workers.

Note

O objetivo desta aula é reconhecer a estrutura do problema. Ainda não é necessário implementar paralelismo ou comunicação entre processos.

1.5 2. Conhecer os dados

Execute:

```
pwd
ls -l
cat ../data/sample.txt
wc -l ../data/sample.txt
wc -w ../data/sample.txt
```

O ficheiro contém:

```
Linux has processes.
A process uses memory; Linux uses files.
Processes use memory and files.
```

Para esta aula, uma palavra é uma sequência de letras ASCII ou algarismos. Pontuação e espaços são separadores. As letras maiúsculas passam para minúsculas.

Por exemplo:

```
"Linux's PROCESS, v2!"
```

torna-se:

```
linux
s
process
v2
```

Usamos LC_ALL=C para que a ordenação seja igual nos ambientes suportados:

```
export LC_ALL=C
```

1.6 3. Construir o pipeline Linux

Comece por separar os caracteres que não pertencem a palavras:

```
tr -cs '[:alnum:]' '\n' < ../data/sample.txt
```

Converta para minúsculas:

```
tr -cs '[:alnum:]' '\n' < ../data/sample.txt |  
tr '[:upper:]' '[:lower:]'
```

Agora remova linhas vazias e ordene:

```
tr -cs '[:alnum:]' '\n' < ../data/sample.txt |  
tr '[:upper:]' '[:lower:]' |  
sed '/^$/d' |  
sort
```

Conte as palavras iguais:

```
tr -cs '[:alnum:]' '\n' < ../data/sample.txt |  
tr '[:upper:]' '[:lower:]' |  
sed '/^$/d' |  
sort |  
uniq -c
```

`uniq -c` só conta ocorrências consecutivas. Por isso `sort` tem de aparecer antes dele.

Para produzir três resultados ordenados por frequência:

```
{  
    printf 'word,count\n'  
    tr -cs '[:alnum:]' '\n' < ../data/sample.txt |  
    tr '[:upper:]' '[:lower:]' |  
    sed '/^$/d' |  
    sort |  
    uniq -c |  
    awk '{ print $2 ", " $1 }' |  
    sort -t, -k2,2nr -k1,1 |  
    head -n 3  
}
```

O resultado deve ser:

word,count
files,2
linux,2
memory,2

Identifique no pipeline:

Parte	Comandos
map	tr
group/shuffle	sort
reduce	uniq -c
seleção final	segundo sort e head

Guarde o resultado num ficheiro, se quiser inspecioná-lo depois:

```
./wordcount.sh ../data/sample.txt 3 > resultado.csv  
cat resultado.csv
```

O último comando ainda não funcionará antes de completar o script.

1.7 4. Automatizar o pipeline com Bash

Abra o starter:

```
sed -n '1,220p' wordcount.sh
```

A validação de argumentos já está preparada. O seu trabalho é substituir o bloco TODO pelo pipeline.

O script deve aceitar:

```
./wordcount.sh ../data/sample.txt  
./wordcount.sh ../data/sample.txt 3
```

Requisitos:

- o primeiro argumento é o ficheiro de entrada;

- o segundo argumento é opcional e indica quantos resultados mostrar;
- o valor predefinido é 10;
- a saída CSV vai para `stdout`;
- mensagens de erro vão para `stderr`;
- ficheiro inexistente e `top-n` inválido devem provocar falha.

O pipeline precisa de usar a variável `TOP_N`, não um número escrito diretamente no comando `head`.

Tip

Construa o pipeline primeiro no terminal. Depois copie as etapas para o script e substitua o número fixo pela variável `TOP_N`.

Teste:

```
./wordcount.sh ../data/sample.txt 3
./wordcount.sh ../data/case_punctuation.txt
./wordcount.sh ../data/empty.txt
```

Teste também os erros:

```
./wordcount.sh ../data/missing.txt
echo $?

./wordcount.sh ../data/sample.txt 0
echo $?
```

O código de saída deve ser diferente de zero nos dois casos de erro.

1.8 5. Reimplementar em C

O programa C chama-se `mr-wordcount` e tem a mesma interface:

```
./mr-wordcount ../data/sample.txt
./mr-wordcount ../data/sample.txt 3
```

Antes de editar, compile o starter:

`make`

```
./mr-wordcount ../data/sample.txt 3
```

O starter já contém o código para:

- interpretar `argc` e `argv`;
- abrir o ficheiro;
- ler e normalizar tokens ASCII;
- criar os pares (`word`, `1`);
- ordenar os pares por palavra;
- ordenar e imprimir o resultado final.

Falta implementar `reduce_sorted`.

Warning

Não altere a tokenização nem a ordenação já fornecidas. A saída C precisa de seguir exatamente o mesmo contrato da versão Bash.

1.8.1 5.1 O tipo usado

```
typedef struct {  
    char word[64];  
    unsigned count;  
} WordCount;
```

Depois do map e da ordenação, pode existir uma sequência como:

```
linux 1  
linux 1  
memory 1  
process 1  
process 1
```

A função deve criar:

```
linux 2  
memory 1  
process 2
```

1.8.2 5.2 Pseudocódigo

Use este algoritmo:

```
reduced_count = 0

para cada entrada de mapped:
    se ainda não existe palavra no resultado
    ou a palavra mudou:
        verificar se há espaço em reduced
        copiar a palavra para uma nova entrada
        iniciar a contagem
        incrementar reduced_count
    caso contrário:
        somar a contagem à última entrada

devolver reduced_count
```

Não altere a assinatura da função. O array mapped já está ordenado por palavra quando `reduce_sorted` é chamada.

Depois de implementar:

```
make clean
make
./mr-wordcount ../data/sample.txt 3
```

O resultado deve ser igual ao do script Bash.

1.9 6. Make e validação

O Makefile compila com warnings ativados e inclui informação para o debugger:

```
make
```

Se alterar o C, execute novamente make. Para remover o executável:

```
make clean
```

Execute a validação completa:

```
make test
```

O teste compara automaticamente a saída Bash com a saída C. Uma execução correta termina com:

```
all tests passed
```

Também pode fazer uma comparação manual:

```
./wordcount.sh ../data/sample.txt 10 > /tmp/bash-result.csv  
./mr-wordcount ../data/sample.txt 10 > /tmp/c-result.csv  
diff -u /tmp/bash-result.csv /tmp/c-result.csv
```

Se `diff` não imprimir nada, as saídas são iguais.

Note

Uma saída vazia do `diff` é o resultado esperado. O `diff` só escreve algo quando encontra uma diferença.

1.9.1 Casos cobertos

Os testes verificam:

- resultado predefinido com `top-n = 10`;
- seleção de três resultados;
- desempate alfabético;
- maiúsculas e pontuação;
- ficheiro vazio;
- `top-n = 1`;
- argumentos inválidos e ficheiro inexistente.

1.10 7. Observação opcional com GDB

Se terminar a tarefa principal, experimente:

```
gdb ./mr-wordcount
```

Dentro do GDB:

```
break reduce_sorted
run ../data/sample.txt 3
next
print mapped_count
backtrace
quit
```

O GDB serve para observar a execução. Não é necessário alterar o programa para esta demonstração.

1.11 8. Trabalho individual

Depois do núcleo da aula, pode continuar com:

1. separar o programa em `main.c`, `mapper.c/.h`, `reducer.c/.h` e `parser.c/.h`;
2. atualizar o Makefile para compilar vários ficheiros `.o`;
3. criar um bug de limite de array e encontrá-lo com GDB;
4. adicionar estatísticas em `stderr` sem alterar o CSV;
5. criar um novo ficheiro de teste;
6. adaptar a leitura para vários ficheiros.

Não implemente ainda processos, threads ou IPC. Esses mecanismos serão introduzidos nas aulas seguintes.

1.12 Checklist

- [] Executei o pipeline manualmente.
- [] Sei explicar `map`, `group/shuffle` e `reduce`.
- [] Completei `wordcount.sh`.
- [] Completei `reduce_sorted`.
- [] `make` termina sem warnings.
- [] `make test` termina com "all tests passed".
- [] Sei explicar por que `sort` aparece antes de `uniq -c`.
- [] Comparei Bash e C no mesmo dataset.